Containerized Real-Time Stock Prediction System

MSML605 — Progress Report

Group Members

1. Dhanvin Suresha Reddy (UID - 121441027)
2. Sarthak Garg (UID - 122275270)
3. Basil Varghese (UID - 122279322)

Problem Statement

Machine learning models deployed in financial markets quickly degrade due to changing market conditions. These changes appear as:

- **Data drift**, where the input feature distribution shifts over time
- **Concept drift** where relationships between inputs and predictions change

This project aims to build a robust, end-to-end machine learning pipeline that not only predicts short-term stock price movements but also actively monitors the health of the model in production.

By integrating a dedicated data drift detection mechanism, the system will automatically flag when shifting market dynamics outdate the model's training data, ensuring long-term reliability and adaptability.

Each Member's Planned Contribution

Dhanvin S : Cloud & Infrastructure Engineer

- **AWS Environment & Security:** Provision the foundational AWS infrastructure (Security Groups, IAM roles)
- **Containerization:** Dockerize the inference engine and training services, ensuring they are optimized for cloud deployment.
- **Deployment & Scaling:** Deploy the model via **SageMaker real-time endpoints** and perform **Locust load testing** to conduct scalability experiments.
- **Automation:** Implement **AWS Lambda** functions to trigger automated retraining or system alerts based on infrastructure health.

Sarthak Garg : Machine Learning Lead

- **Feature Engineering:** Build the preprocessing pipeline for time-series data, calculating technical indicators like RSI, Moving Averages, and Volume trends.
- **Model Development:** Design, train, and hyperparameter-tune **XGBoost** or **LSTM** models specifically for stock price forecasting.
- **Baseline Definition:** Establish the SageMaker Model Monitor baseline by capturing the statistical "signature" of the training data.
- **Quantitative Analysis:** Define and track ML-specific evaluation metrics (like RMSE or MAE) to benchmark model performance against historical data.

Basil Varghese : MLOps & Systems Monitoring Lead

- **Real-time Monitoring:** Configure **SageMaker Data Capture** and **Model Monitor** to track live inference data.
- **Drift Detection:** Develop custom scripts using statistical tests (e.g., Kolmogorov-Smirnov) to identify feature degradation and data drift.
- **Observability Dashboard:** Build a **Streamlit/Dash** frontend that visualizes live predictions vs. actual prices alongside real-time drift metrics.
- **System Metrics:** Configure **CloudWatch Alarms** for both system health (CPU/Memory) and ML health (Drift/Accuracy), collecting data for the final performance report.

High-Level Architecture

1. **Infrastructure & Deployment:** The core prediction API and monitoring services will be packaged within Docker containers and deployed onto AWS ECR. This ensures a consistent, isolated environment from development to production.
2. **Data Ingestion:** Real-time and historical financial data will be pulled via APIs (such as Defeat Beta API, Yahoo Finance, or Alpha Vantage).
3. **Modeling:** Time-series forecasting models (such as LSTMs or XGBoost) will be used to predict closing prices.
4. **MLOps & Monitoring:** A continuous monitoring service will run alongside the prediction model and track data drift using statistical tests (KS statistic / PSI) to monitor feature degradation.

Novelty

Unlike typical machine learning projects, our work focuses on **deployment, automation, and quantitative infrastructure evaluation rather than only predictive accuracy**.

Our novelty lies in:

- Fully containerized ML deployment in the cloud
- Automated drift-aware retraining loop
- System-level quantitative evaluation of MLOps benefits
- Demonstration of scalability and reliability under real-time streaming load

The project emphasizes the operational value of containerized ML systems.

Implementation Tools

Cloud Services

- Defeat Beta API, Yahoo Finance, or Alpha Vantage API
- Amazon SageMaker (Training, Endpoint, Model Monitor)
- Amazon S3
- Amazon CloudWatch
- Amazon Lambda
- Amazon ECR
- Amazon EventBridge
- Amazon SNS

Containerization & DevOps

- Docker
- GitHub Actions (for CI/CD)

Machine Learning Stack

- XGBoost or PyTorch
- Scikit-learn
- Pandas / NumPy

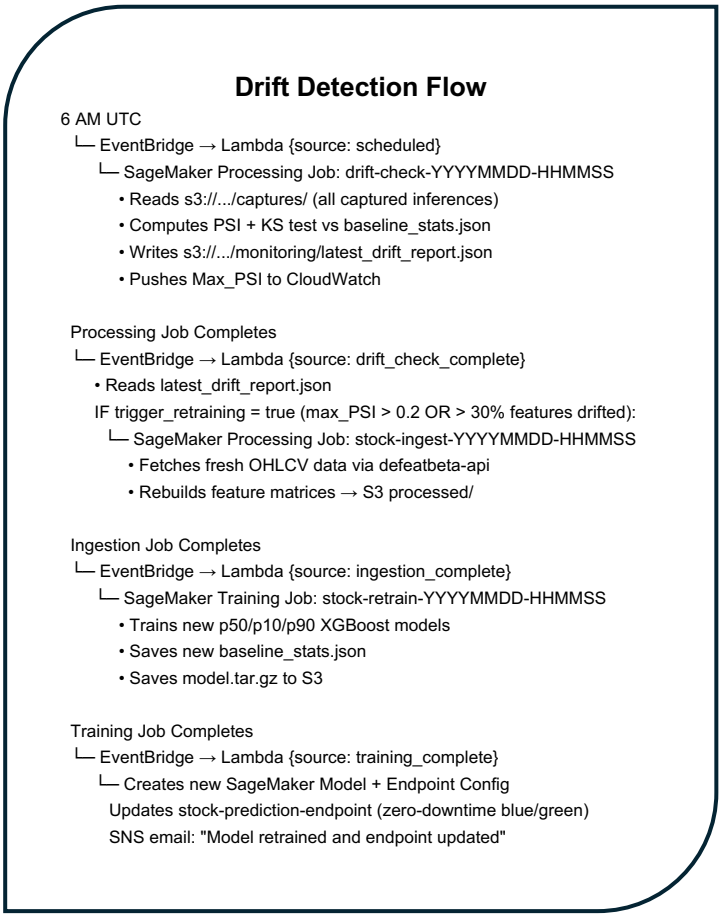
Evaluation Tools

- Locust (load testing)
- CloudWatch metrics
- Statistical drift metrics (KS test, PSI)

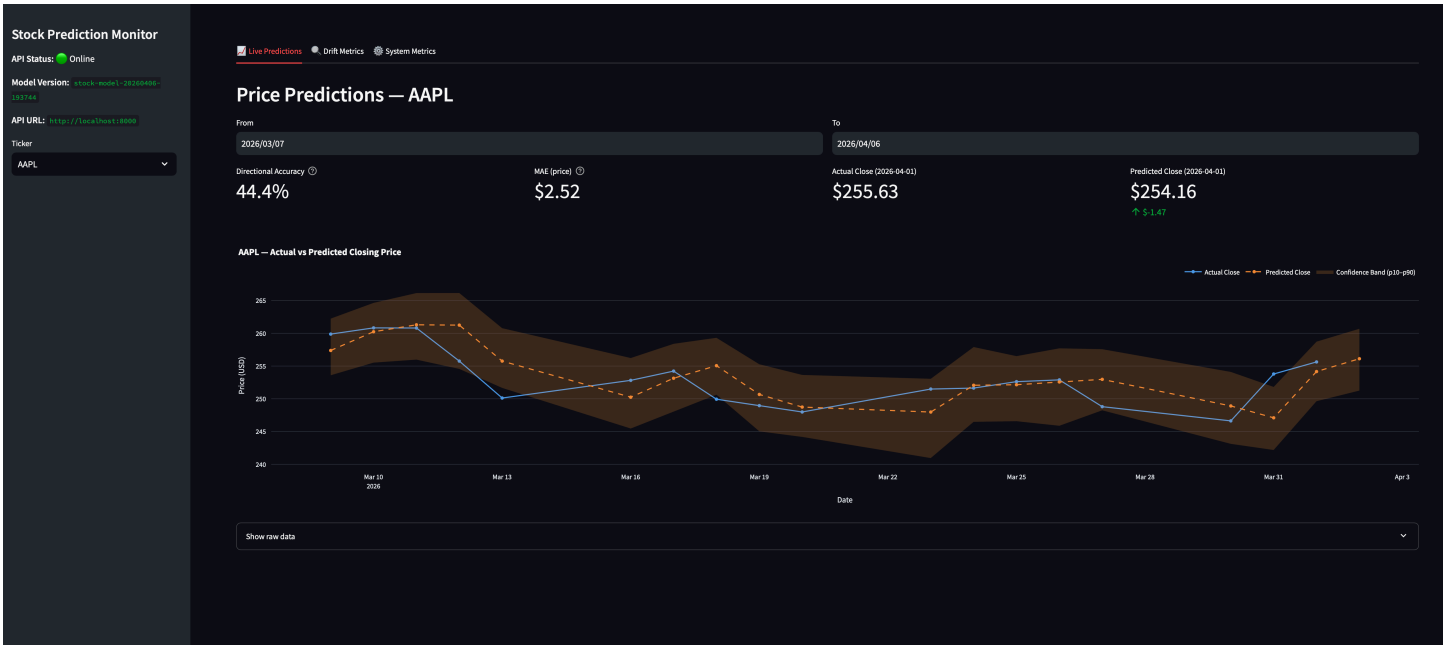
Progress

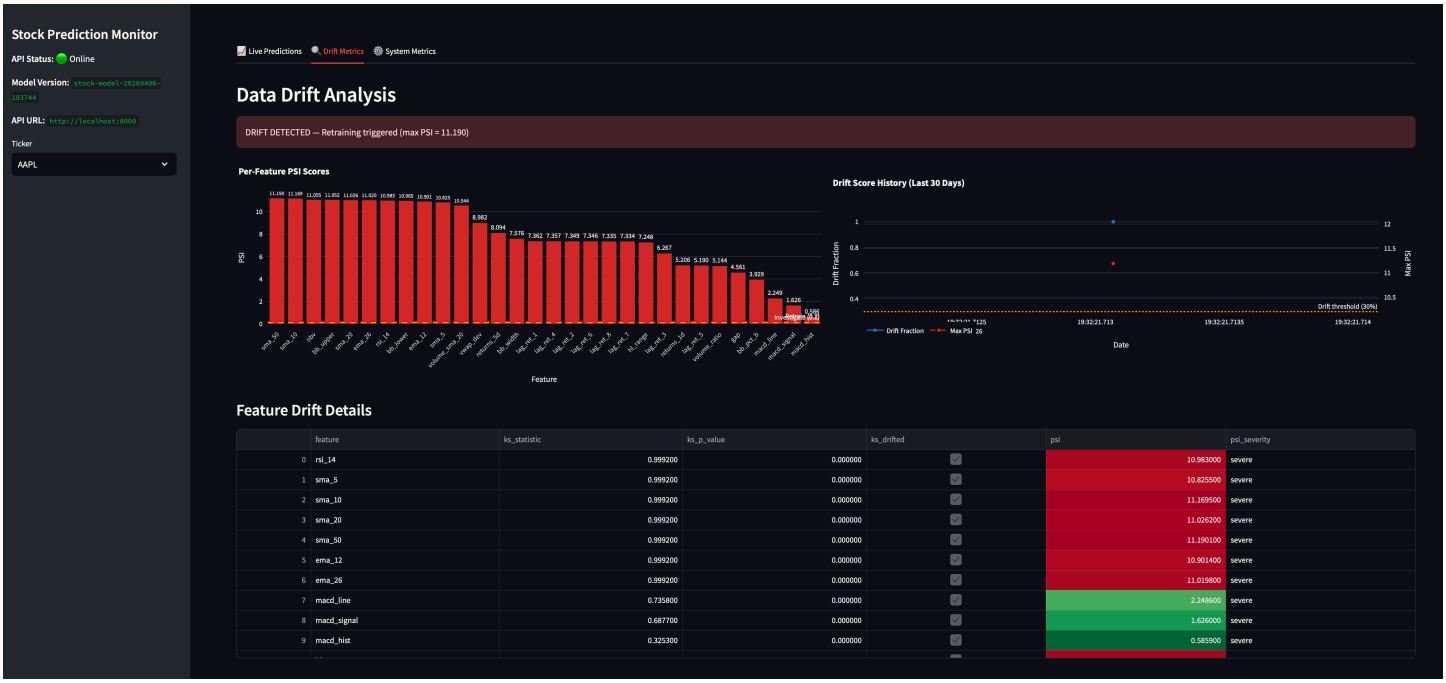
Starting from raw market data, the system cleans and transforms it into 23 technical indicators, trains an XGBoost model, and exposes predictions through Sagemaker inference endpoint. Once deployed, it continuously monitors whether the data the model sees in production still matches what it was trained on — and automatically kicks off retraining if it doesn't.

Component	Description	Status
Data Ingestion	Fetches stock data from Defeat Beta API with Yahoo Finance and Alpha Vantage as fallback	**Complete**
Feature Engineering	23 technical indicators: RSI, moving averages, Bollinger Bands, volume features, and lagged returns	**Complete**
Model Training	XGBoost with walk-forward cross-validation to prevent look-ahead leakage; quantile outputs for prediction intervals	**Complete**
Inference API	Sagemaker inference endpoint serving predictions with low/mid/high confidence bounds	**Complete**
Drift Detection	Kolmogorov-Smirnov test and Population Stability Index run once a day on live inference data vs training baseline	**Complete**
Auto-Retraining	CloudWatch alarm triggers Lambda which launches a new SageMaker training job when drift is detected	**Complete**
Dashboard	Streamlit app with three tabs: live predictions, drift metrics, and system health	**Complete**
Containerisation	Docker images for training, inference, and dashboard deployed on AWS ECR	**Complete**
CI/CD Pipeline	GitHub Actions workflow — lint, unit tests, Docker build, push to ECR	**In Progress**
Load Testing	Locust scenarios covering normal use, stress test	**In Progress**



Dashboard





	feature	ks_statistic	ks_p_value	ks_drifted	psi	psi_severity
0	rsi_14	0.999200	0.000000	☑	10.983000	severe
1	sma_5	0.999200	0.000000	☑	10.825500	severe
2	sma_10	0.999200	0.000000	☑	11.169500	severe
3	sma_20	0.999200	0.000000	☑	11.026200	severe
4	sma_50	0.999200	0.000000	☑	11.190100	severe
5	ema_12	0.999200	0.000000	☑	10.901400	severe
6	ema_26	0.999200	0.000000	☑	11.019800	severe
7	macd_line	0.735800	0.000000	☑	2.248600	severe
8	macd_signal	0.687700	0.000000	☑	1.626000	severe
9	macd_hist	0.325300	0.000000	☑	0.585900	severe

Stock Prediction Monitor

API Status: Online

Model Version: `stock-model-20260406-193744`

API URL: `http://localhost:8000`

Ticker

AAPL

Live Predictions | Drift Metrics | System Metrics

System & Infrastructure Metrics

Total Requests	Avg Latency	p95 Latency	Uptime
532	41.8 ms	115.4 ms	1.5 h

API Latency Over Time

Retraining Event History

	timestamp	event
0	2026-04-06 19:37 UTC	training_complete
1	2026-04-06 19:38 UTC	training_complete
2	2026-04-06 19:40 UTC	training_complete
3	2026-04-06 19:30 UTC	scheduled
4	2026-04-06 19:33 UTC	drift_check_complete
5	2026-04-06 19:35 UTC	ingestion_complete
7	2026-04-06 19:17 UTC	scheduled